

# Tema 7:

# Conjuntos y bolsas

GRADO EN INGENIERÍA DEL SOFTWARE

*Salvador Sánchez y Sergio Caverio*



Universidad  
Rey Juan Carlos

Curso 2025-2026

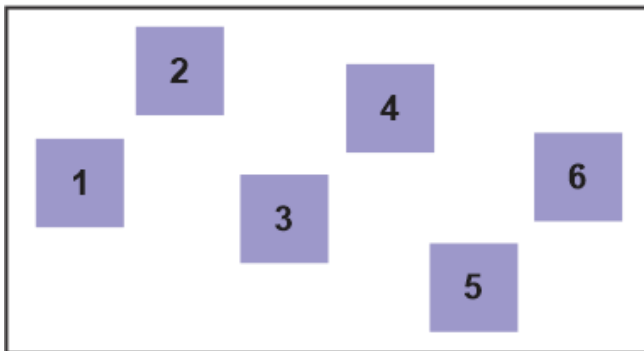
# Contenidos

- Definición
- Los TADs Conjunto y Bolsa
- Conjuntos en Pascal
- Implementación
  - Arrays vs. memoria dinámica
  - Tablas Hash
  - BST
- Resumen

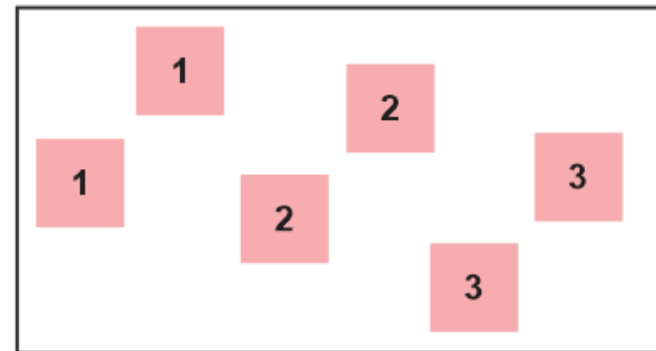
# Conjunto y bolsa: definiciones

- **Conjunto (set):** Estructura de datos no lineal que representa una colección no ordenada de elementos en la que no se admiten duplicados.
- **Bolsa (multiset):** contenedor similar a un conjunto que admite duplicados.

Set:



Multiset:

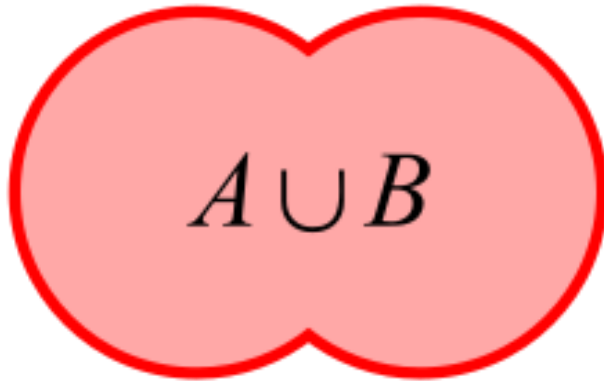


# TAD Conjunto

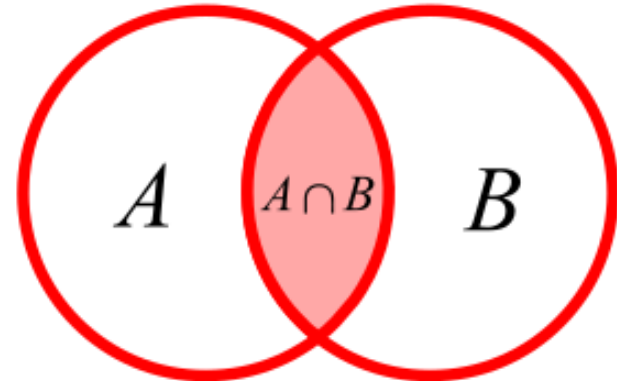
- Independientemente de su implementación, un conjunto proporciona las siguientes 4 operaciones básicas:
  - Añadir un elemento (add)
  - Eliminar un elemento (remove)
  - Consultar si un elemento está en el conjunto (in)
  - Conocer el tamaño o número de elementos (size)
- + Inicializar/crear
- Adicionalmente: Unión, Intersección, Diferencia, Extracción de subconjuntos, Inclusión, etc.

# TAD Conjunto

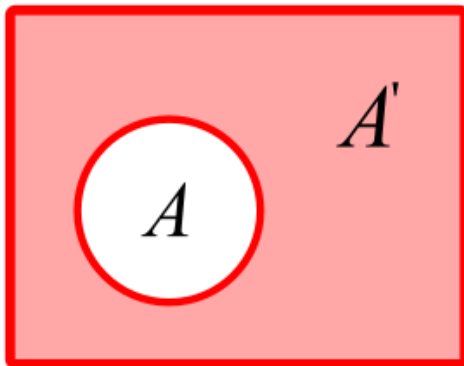
Unión



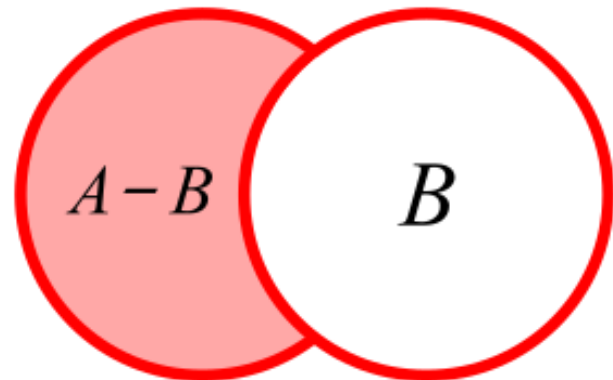
Intersección



Complemento



Diferencia



# TAD Conjunto: especificación

- **Nombre:** Conjunto
- **Valores posibles:** *vacío, otros (según elementos).*
- **Operaciones:**
  - add
  - remove
  - in
  - size
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)

# TAD Bolsa

- Una bolsa proporciona 5 operaciones básicas:
  - Añadir un elemento (add)
  - Eliminar un elemento (remove)
  - Eliminar todas las ocurrencias de un elemento (remove\_all)
  - Multiplicidad de un elemento (multiplicity)
  - Conocer el tamaño o número de elementos (size)
- + Inicializar/crear

# TAD Bolsa: especificación

- **Nombre:** Bolsa
- **Valores posibles:** *vacía, otros (según elementos).*
- **Operaciones:**
  - add
  - remove
  - remove\_all
  - multiplicity
  - size
- **Uso de otros tipos:** tipo base, arrays / punteros (según implementación)



# Implementación

- Los conjuntos y bolsas pueden implementarse utilizando diversas estructuras de datos, como arrays, listas enlazadas, árboles de búsqueda binarios o tablas hash.
- Implementación en memoria contigua (arrays):
  - Acceso eficiente e inmediato a cualquier elemento mediante acceso aleatorio.
  - El tamaño fijo puede llevar a quedarse sin espacio o a desperdiciarlo.
- Implementación en memoria dinámica:
  - Almacenamiento eficiente y ajustado al número de nodos
  - Encontrar un nodo concreto requiere recorridos de (potencialmente) muchos otros nodos precedentes  $\rightarrow O(n) / O(\log n)$

# Conjuntos en Pascal

- Pascal incluye una implementación de conjuntos muy simple y limitada, pero útil en algunos casos.
- El tipo **Set** permite conjuntos homogéneos cuyo tipo base es cualquier ordinal (incluidos enumerados)
- Todo conjunto en Pascal está limitado a 256 elementos

| Operación                 | Operador                                    |
|---------------------------|---------------------------------------------|
| Unión                     | <code>+</code>                              |
| Intersección              | <code>*</code>                              |
| Diferencia                | <code>-</code>                              |
| Diferencia simétrica      | <code>&gt;&lt;</code>                       |
| Igualdad                  | <code>=</code>                              |
| Desigualdad               | <code>&lt;&gt;</code>                       |
| Inclusión                 | <code>&lt;=</code> , <code>&gt;=</code>     |
| Pertenencia               | <code>in</code>                             |
| Añadir/eliminar elementos | <code>include</code> / <code>exclude</code> |

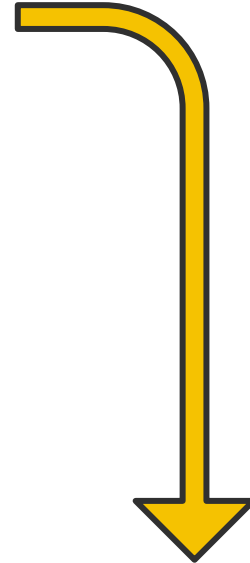
# Conjuntos en Pascal

- Operaciones básicas:

```
type
    tipo_base = (value1, value2, ... value_n);
    tSet = set of tipo_base;

var
    my_set: tSet;

begin
    my_set := [value_i, value_j...];
    if some_value in my_set then do_something
end.
```



*Como tipo base sirve cualquier ordinal cuyo rango de valores sea máximo 256*

# Conjuntos en Pascal

- Ejemplo:

```
type
    language = (solidity, java, python, C++, ruby, go);
    languages = set of language;

var
    programmer_skills: languages;

begin
    programmer_skills:= [java, python];
    if solidity in programmer_skills then
        writeln('The guy is a blockchain developer!');
end.
```

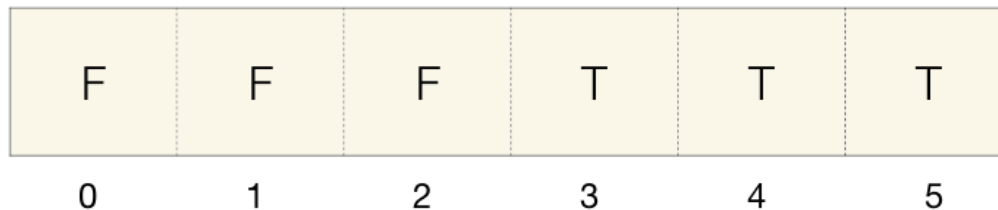
# Conjuntos en Pascal

- Otro ejemplo:

```
type
    conjunto_caracteres = set of char;
var
    s1, s2: conjunto_caracteres;
begin
    s1 := ['1', '2', '3', '4'];
    s2 := ['3', '4', '5', '6'];
    include(s1, 'X');
    exclude(s1, '2');
    writeln('X' in s1);
    readln;
end.
```

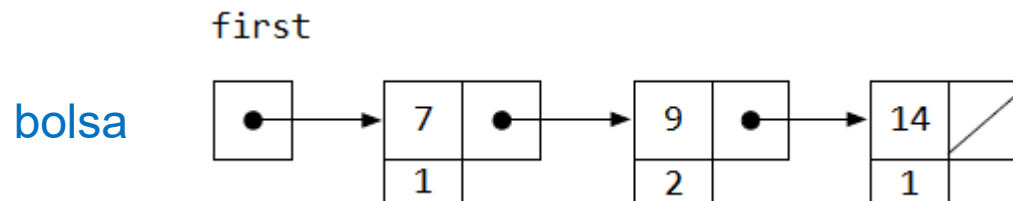
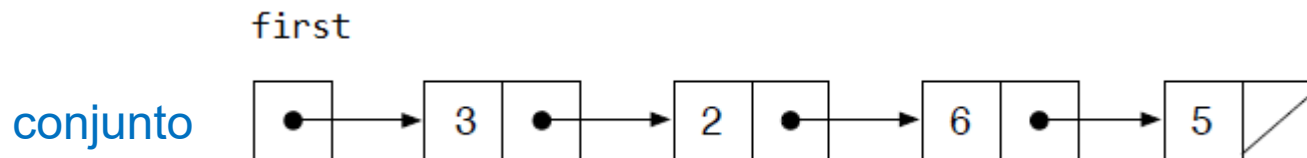
# Implementación mediante arrays

- Se define un array cuyos índices son los posibles valores que pueden incluirse en el conjunto.
- Limitado a tipos base que permitan indexar un array: generalmente enteros y otros ordinales.
  - Este es el enfoque del tipo Set en Pascal



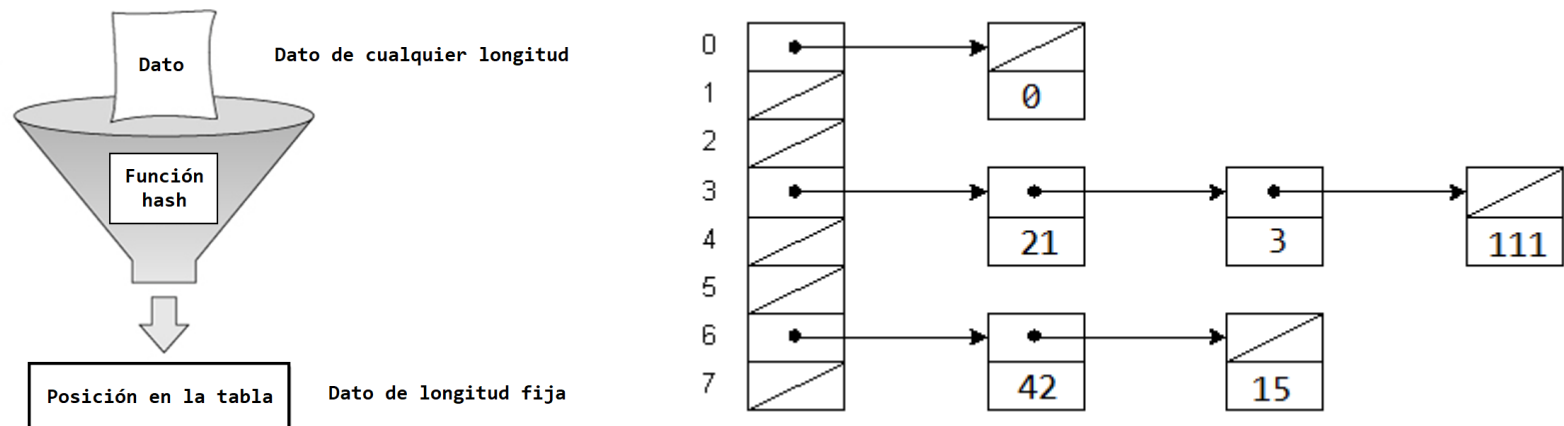
# Implementación mediante listas

- Se utiliza una lista (ordenada o no) para almacenar los valores del conjunto.
  - Todo elemento en la lista, está en el conjunto
  - En bolsas, hay un campo adicional para la multiplicidad



# Implementación mediante tablas hash

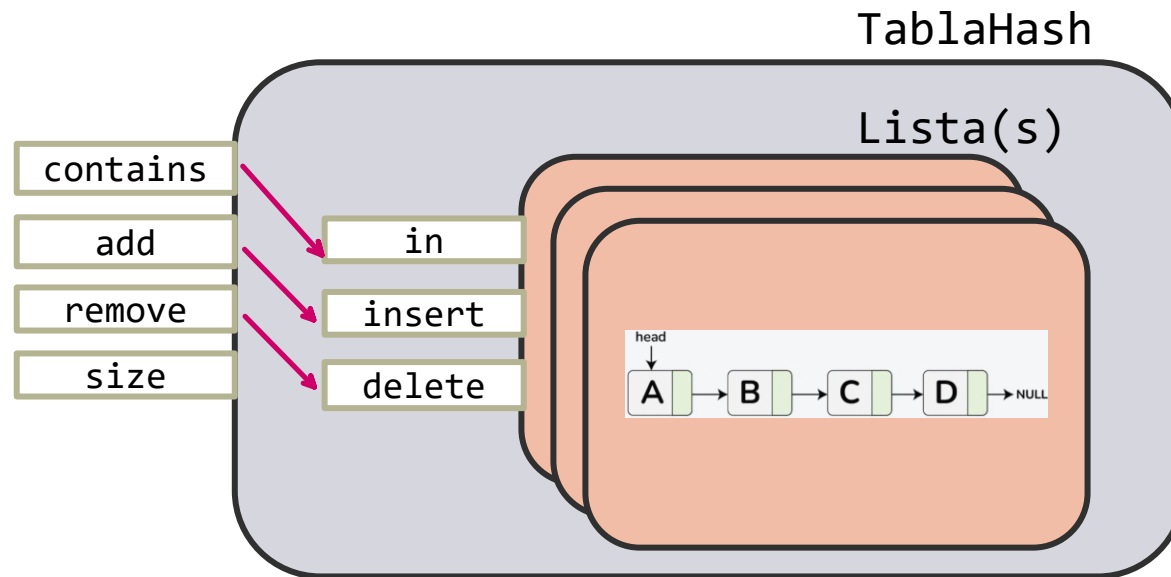
- Una función hash facilita un acceso eficiente a la lista de claves
- Como puede haber colisiones (elementos con la misma clave hash) en cada valor de la clave hay una lista enlazada con todos los valores cuyo hash es el mismo.





# Implementación mediante tablas hash

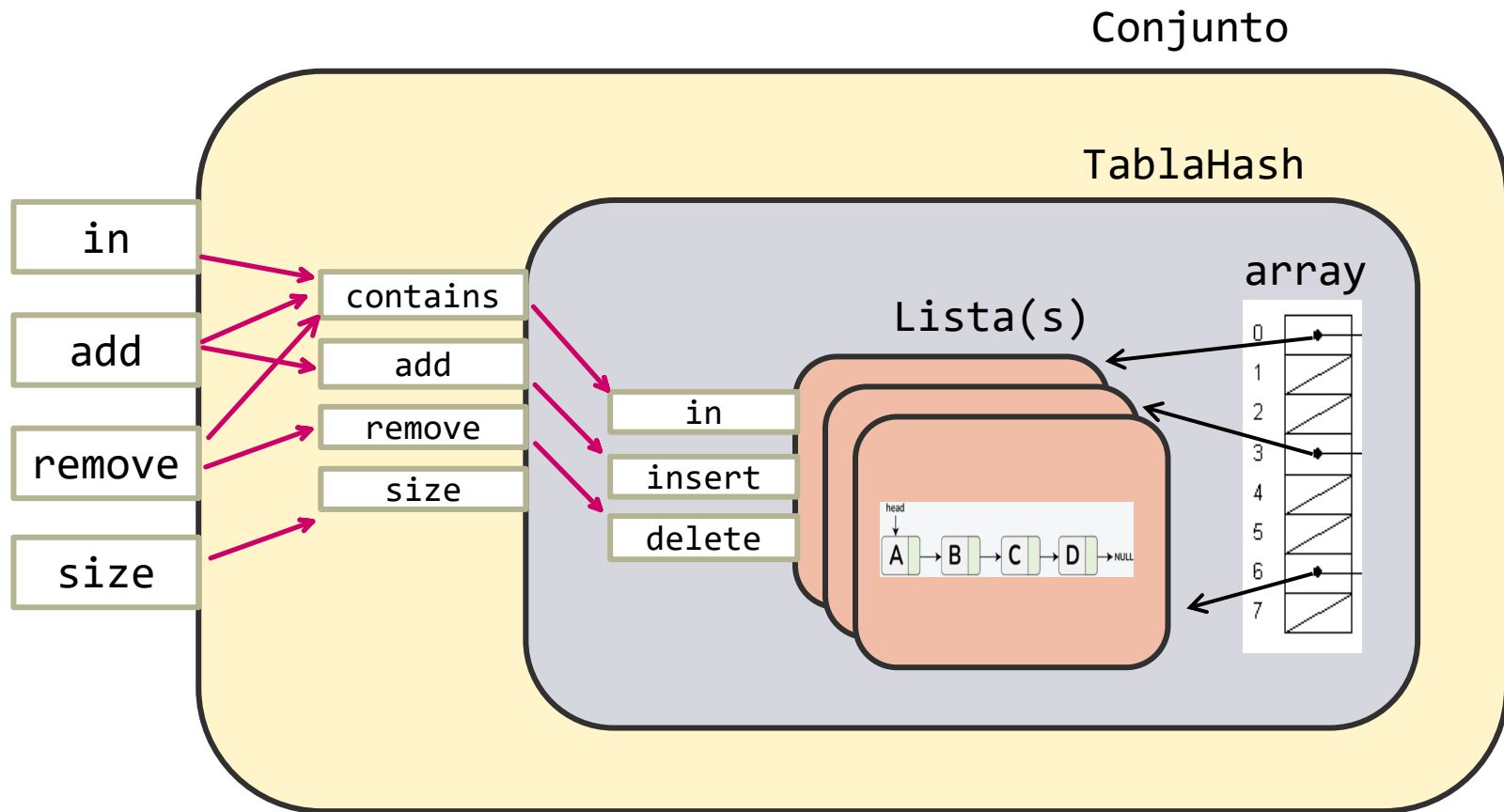
- La implementación de una TablaHash utiliza el TAD ListaEnlazada como usuario de dicho TAD pues:
  - Cada vez que se añade a la TablaHash un elemento lo que ocurre internamente es que se añade a una de las listas de la tabla
  - Algo similar ocurre al eliminar elementos y al comprobar si un elemento está



# Implementación mediante tablas hash

- La implementación de un Conjunto con una TablaHash interna usa el TAD TablaHash pues...
  - Cada vez que se añade un elemento al Conjunto se busca en la TablaHash y si no está se añade a la tabla.
  - Cada vez que se desea eliminar un elemento del Conjunto se busca en la TablaHash y si está se elimina de la misma
  - Cada vez que se pregunta si un elemento está en el Conjunto se busca si está en la TablaHash y según el resultado se retornará verdadero o falso

# Implementación mediante tablas hash



# Implementación de una tabla *hash*

```
unit uHashTable;

interface
  uses
    sysUtils, uListaEnlazadaSimple; // Simple o de otro tipo
  const
    HASHTABLESIZE = 10;
  type
    tHashTable = array[0..HashTableSize-1] of tListaSimple;

  procedure initialize(var table: tHashTable);
  procedure add(var table: tHashTable; key: string);
  procedure remove(var table: tHashTable; key: string);
  function contains(table: tHashTable; key: string): boolean;

implementation
  function hash_function(key: string): integer; // Uso interno solo
  // resto de implementación
```

# Implementación de una tabla hash

```
function hash_function(key: string): integer;  
var  
    i, hash: integer;  
begin  
    hash := 0;  
    for i := 1 to length(key) do  
        hash := (hash + ord(key[i]));  
    hash_function := hash mod HASHTABLESIZE;  
end;
```

# Implementación mediante tablas hash

```
procedure initialize(var table: tHashTable);  
var  
    i: integer;  
begin  
    for i := 0 to HASHTABLESIZE - 1 do  
        uListaEnlazadaSimple.initialize(table[i]);  
    end;
```

# Implementación mediante tablas hash

```
procedure add(var table: tHashTable; key: string);  
var  
    index: integer;  
begin  
    index := hash_function(key);  
    if not uListaEnlazadaSimple.in_list(table[index], key) then  
        begin  
            uListaEnlazadaSimple.insert_at_end(table[index], key);  
        end;  
    end;  
end;
```

# Implementación mediante tablas hash

```
procedure remove(var table: tHashTable; key: string);  
var  
    index: integer;  
begin  
    index := hash_function(key);  
    uListaEnlazadaSimple.delete(table[index], key);  
end;
```



# Implementación mediante tablas hash

```
function contains(table: tHashTable; key: string): Boolean;  
var  
    index: integer;  
begin  
    index := hash_function(key);  
    contains := uListaEnlazadaSimple.in_list(table[index], key);  
end;
```

# Implementación mediante tablas hash

```
procedure show_table_state(table: tHashTable);  
var  
    i: integer;  
begin  
    for i := 0 to HASHTABLESIZE - 1 do begin  
        write('Index ', i, ': ');  
        writeln(uListaEnlazadaSimple.to_string(table[i]));  
    end;  
end;
```

# La unidad Conjunto en Pascal

```
unit uConjunto;

interface
  uses
    SysUtils, uHashTable;
  type
    TConjunto = record
      table_hash: tHashTable;
      num_elems: integer;
    end;
  procedure initialize(var c: TConjunto);
  procedure add(var c: TConjunto; clave: string);
  procedure remove(var c: TConjunto; clave: string);
  function is_in(c: TConjunto; clave: string): boolean; // No es válido "in"
  function size(c: TConjunto): integer;
```

# La unidad Conjunto en Pascal

```
procedure initialize(var c: TConjunto);  
begin  
    uHashTable.initialize(c.table_hash);  
    c.num_elems := 0;  
end;
```

# La unidad Conjunto en Pascal

```
function is_in(c: TConjunto; clave: string): boolean;  
begin  
    is_in := uHashTable.contains(c.table_hash, clave);  
end;
```

```
function size(c: TConjunto): integer;  
begin  
    size := c.num_elems;  
end;
```

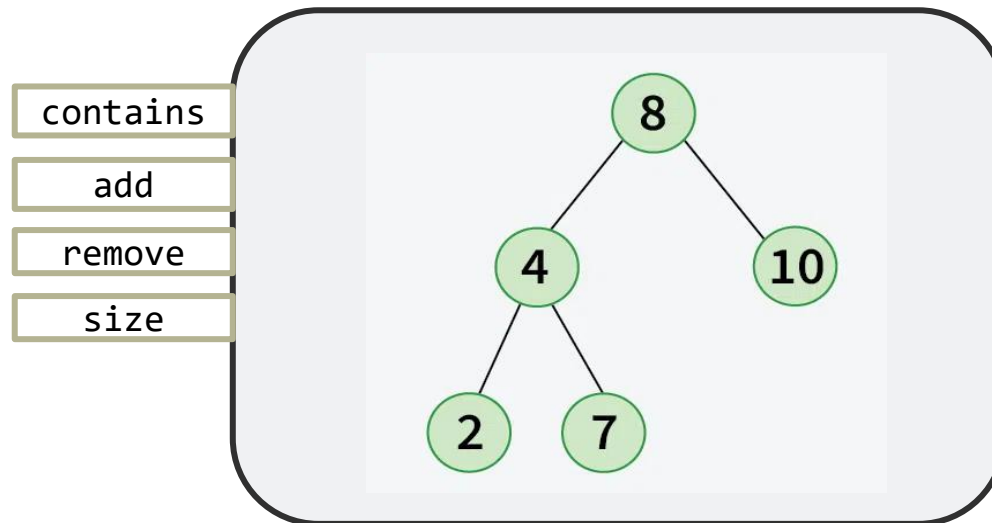
# La unidad Conjunto en Pascal

```
procedure add(var c: TConjunto; clave: string);  
begin  
    uHashTable.add(c.table_hash, clave);  
    inc(c.num_elems);  
end;
```

```
procedure remove(var c: TConjunto; clave: string);  
begin  
    uHashTable.remove(c.table_hash, clave);  
    dec(c.num_elems);  
end;
```

# Implementación mediante un BST

- Solución intermedia entre la eficiencia de acceso de los arrays y la flexibilidad de las listas
  - Acceso más eficiente: búsquedas  $O(\log(n))$
  - Elementos siempre ordenados
  - Solución dinámica y flexible en cuanto a tipo de elementos



# Resumen

- Aunque estructuras similares, existe una diferencia esencial entre un conjunto y una bolsa: el conjunto no permite duplicados mientras que en la bolsa puede haber varias instancias de un mismo elemento
- Los elementos no están ordenados, simplemente pertenecen o no al conjunto (o bolsa) o no pertenecen
- Una de las aplicaciones más comunes de conjuntos/bolsas es la eliminación de elementos duplicados en otro almacenamiento
- Pascal proporciona un soporte muy básico y limitado al concepto de conjunto mediante el tipo Set
- Los TADs conjunto y bolsa son apenas una interfaz que internamente almacena los elementos en un contenedor “propriadamente dicho”: una lista (ordenada o no), un árbol, una tabla hash...
- Las operaciones de la interfaz de un conjunto invocan a menudo a las de su contenedor interno sin entrar en detalles de cómo éste está implementado: excelente ejemplo del concepto de encapsulación